

การพัฒนาและประยุกต์ใช้งาน

Shared Memory (POSIX SHM) บน Linux

1. บทนำและแนวคิดพื้นฐานของ POSIX Shared Memory

POSIX Shared Memory (SHM) เป็นกลไกการสื่อสารระหว่างโปรเซส (Inter-Process Communication - IPC) บนระบบปฏิบัติการ Linux ที่มีประสิทธิภาพสูงที่สุด โดยอนุญาตให้หลายโปรเซส (แม้จะพัฒนาด้วยภาษาโปรแกรมที่แตกต่างกัน) สามารถเข้าถึงและมองเห็น "หน่วยความจำก้อนเดียวกัน" (Shared Memory Region) ผ่านทาง Virtual File System ที่ถูกแมปไว้ที่ไดเรกทอรี `/dev/shm`

หลักการทำงานพื้นฐานประกอบด้วยขั้นตอนสำคัญ 5 ประการ ได้แก่:

- **1. เปิด/สร้างไฟล์:** โปรเซสแรกเรียกใช้ฟังก์ชัน `shm_open()` เพื่อสร้างหรือเปิดไฟล์พิเศษที่อยู่ใน `/dev/shm`
- **2. กำหนดขนาด:** กำหนดขนาดของหน่วยความจำแชร์ที่ต้องการด้วยฟังก์ชัน `ftruncate()`
- **3. แมปหน่วยความจำ:** เรียกฟังก์ชัน `mmap()` เพื่อแมป (Map) ไฟล์ดังกล่าวเข้ามาเป็นพื้นที่หน่วยความจำใน Virtual Address Space ของโปรเซสตนเอง
- **4. การเข้าถึงร่วมกัน:** โปรเซสอื่นๆ สามารถเปิดไฟล์ชื่อเดียวกันด้วย `shm_open()` และเรียก `mmap()` เพื่อเข้าถึงข้อมูลชุดเดียวกันแบบ Real-time โดยไม่มีการทำสำเนาข้อมูล (Data Copying) ข้ามโปรเซสเหมือนกลไก Socket หรือ Pipe
- **5. การปิดและทำลาย:** เมื่อเสร็จสิ้นการทำงาน โปรเซสจะคืนทรัพยากรด้วย `munmap()` และสามารถลบชื่อ SHM ออกจากระบบด้วย `shm_unlink()` เมื่อเลิกใช้งานถาวร

ข้อดีหลักของ POSIX SHM คือความเร็วในการทำงานที่สูงมากเนื่องจากไม่มี Overhead จาก System Call
คั่นกลางระหว่างการอ่าน-เขียนแต่ละครั้ง อย่างไรก็ตาม ข้อควรระวังที่สำคัญคือ POSIX SHM
ไม่มีระบบล็อกข้อมูลให้อัตโนมัติ (No Automatic Synchronization)
ผู้พัฒนาจึงต้องออกแบบการซิงโครไนซ์ข้อมูลด้วยตนเอง

2. สถาปัตยกรรมและการเปรียบเทียบรูปแบบการออกแบบโปรเจกต์

ในระบบงานจริงชุดนี้ ได้แบ่งรูปแบบการออกแบบโครงสร้าง Shared Memory ออกเป็น 2 รูปแบบหลัก
เพื่อให้เหมาะสมกับลักษณะของข้อมูลและอัตราความถี่ในการรับส่ง (Data Characteristic)
ดังตารางเปรียบเทียบด้านล่าง:

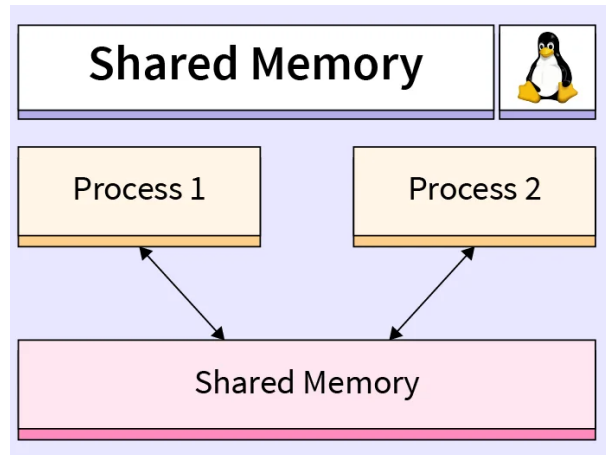
รูปแบบการออกแบบ	ภาษาที่เกี่ยวข้อง	ลักษณะข้อมูล	กลไกการซิงโครไนซ์
1. Struct + Ring Buffer	C/C++	ข้อมูลขนาดเล็ก ยิงถึง ห้ามหลุด (เช่น มุม IMU, พิกัด GPS)	Lock-free SPSC Queue (Single-Producer Single-Consumer)
2. Raw Byte Offset + Flag	Go & C/C++ Python	ข้อมูลก้อนใหญ่ อัปเดตทับตลอด (เช่น ภาพวิดีโอจากกล้อง 640x480)	Atomic Ready Flag + Timestamp (Frame-based synchronization)

3. รายละเอียด Library และ System Calls ที่สำคัญ (C/C++)

การพัฒนาโปรแกรมภาษา C/C++ เพื่อใช้งาน POSIX SHM จำเป็นต้องเรียกใช้งาน Header Files และ System Calls หลัก ดังแสดงในตารางสรุป:

Library / Header	ฟังก์ชันหลัก	คำอธิบายและหน้าที่การทำงาน
<sys/mman.h>	mmap(), munmap()	ใช้สำหรับแมปและยกเลิกการแมปไฟล์ SHM เข้าสู่ Virtual Address Space ของโปรเซส
<fcntl.h>	shm_open()	ใช้สร้างหรือเปิด Object ของ Shared Memory ในระบบ Virtual File System (/dev/shm)
<unistd.h>	ftruncate(), close()	ใช้กำหนดขนาด (Size) ของ SHM และปิด File Descriptor หลังจากการแมปเสร็จสิ้น
<sys/stat.h>	fchmod()	ใช้บังคับสิทธิ์การเข้าถึงไฟล์ SHM (เช่น 0666) ป้องกันปัญหา Umask ตัดสิทธิ์
<atomic>	std::atomic	ใช้จัดการตัวแปรแบบ Atomic สำหรับทำ Ring Buffer แบบ Lock-free ป้องกัน Race Condition

4. ขั้นตอนการสร้างและใช้งาน Shared Memory ฝั่ง C/C++



4.1 ฝั่งเขียน (Writer / Producer)

ในฝั่งของ Writer จะต้องทำหน้าที่สร้างหรือเปิด SHM กำหนดขนาด ตั้งค่าสิทธิ์ และทำ Real-time Scheduling เพื่อให้การอ่านเซนเซอร์มีความแม่นยำสูง โค้ดมาตรฐานจาก shm_common.h มีรายละเอียดดังนี้:

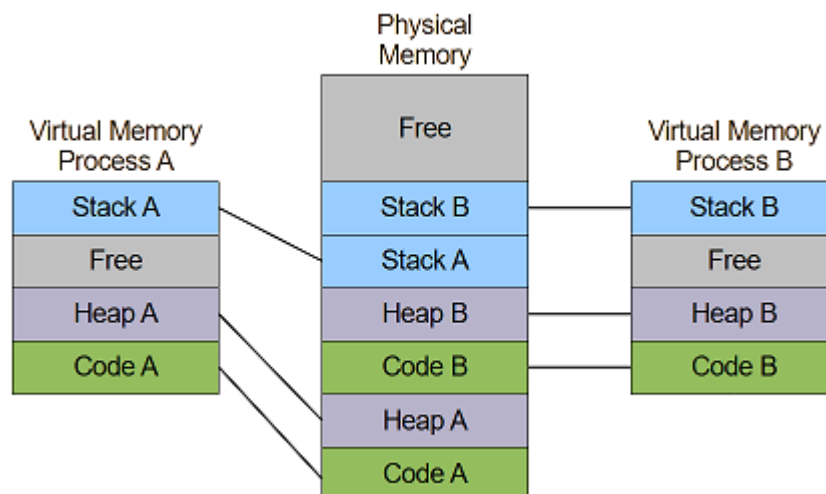
- **1. Real-time Scheduling:** กำหนด Priority ด้วย SCHED_FIFO เพื่อให้ระบบปฏิบัติการให้ความสำคัญกับ Thread นี้เป็นอันดับแรก (ต้องรันด้วยสิทธิ์ root)
- **2. สร้าง/เปิด SHM:** เรียก shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666) เพื่อเปิดหรือสร้าง SHM โดยใช้ Flag O_CREAT เสมอ เพื่อให้สามารถรีสตาร์ท Writer ได้โดยข้อมูลเดิมไม่สูญหาย
- **3. กำหนดขนาดและสิทธิ์:** เรียก fchmod(fd, 0666) และ ftruncate(fd, SHM_SIZE) เพื่อกำหนดขนาดพื้นที่ (เช่น 1MB) และสิทธิ์การเข้าถึง
- **4. แมปหน่วยความจำ:** เรียก mmap() ด้วย Flag MAP_SHARED เพื่อเชื่อมต่อหน่วยความจำ จากนั้นปิด File Descriptor และใช้งาน Pointer ได้ทันที

4.2 ฝั่งผู้อ่าน (Reader / Consumer)

ฝั่ง Reader จะทำหน้าที่เปิด SHM เพื่ออ่านข้อมูลอย่างเดียว (Read-only / Read-write แต่อ่านเป็นหลัก)

โดยมีข้อกำหนดที่สำคัญคือ:

- 1. ห้ามสร้างใหม่: ไม่ใช่ Flag `O_CREAT` หาก Writer ยังไม่รัน `shm_open` จะคืนค่า -1 ทันที ซึ่ง Reader ต้องดักจับ Error ส่วนนี้อย่างสุภาพ
- 2. ห้ามเปลี่ยนขนาด: ไม่เรียก `ftruncate` หรือ `fchmod` เพื่อป้องกันการไปเปลี่ยนแปลงโครงสร้างหรือขนาดที่ Writer สร้างไว้
- 3. การจัดการ CPU Load: ใช้งานฟังก์ชัน `sched_yield()` ใน Loop กรณีที่ไม่มีข้อมูลใหม่ เพื่อป้องกันการกิน CPU 100% จากการทำ Busy-wait Polling



ตัวอย่างภาพ: การ mmap memory

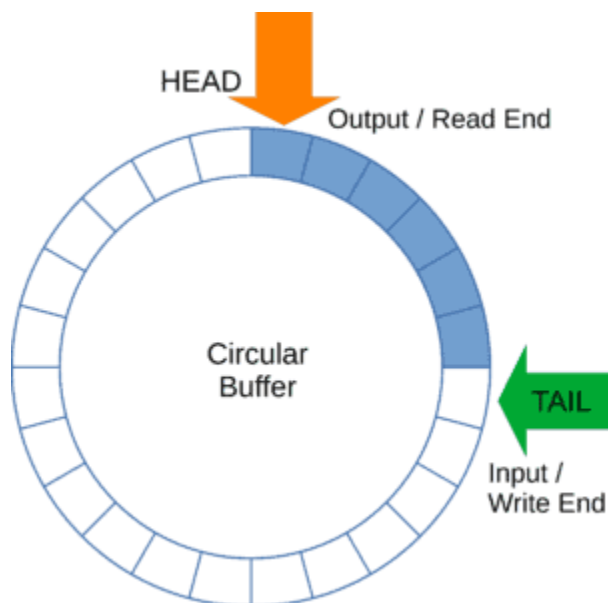
5. กลไกการชิงโครไนซ์แบบ Lock-free SPSC Ring Buffer

สำหรับการส่งข้อมูลขนาดเล็กที่มีความถี่สูง เช่น ข้อมูล IMU และ GPS ทางโปรเจกต์ได้ออกแบบโครงสร้าง Ring Buffer แบบ Single-Producer Single-Consumer (SPSC) โดยใช้ `std::atomic<size_t>`

สำหรับควบคุมตำแหน่ง head และ tail

หลักการทำงานเพื่อความปลอดภัยในกรณีโปรเซสอยู่ต่าง core มีดังนี้:

- **1. Atomic Index:** เก็บตัวแปร head (ตำแหน่งเขียน) และ tail (ตำแหน่งอ่าน) เป็นแบบ atomic ทำให้การอ่านเขียน index ไม่มีทางเกิด Partial Write
- **2. ข้อจำกัด SPSC:** กำหนดให้มี Producer เพียง 1 ตัว และ Consumer เพียง 1 ตัวต่อ 1 Ring Buffer หากมีหลายตัวเขียนพร้อมกันจะเกิด Race Condition
- **3. Memory Ordering:** การใช้ Memory Order แบบ release ตอน push และ acquire ตอน pop ช่วยการรันลำดับการมองเห็นของข้อมูลใน CPU Cache โดยไม่ต้องใช้ Heavy Lock
- **4. การเช็คสถานะ Full/Empty:** หาก `next == tail` แสดงว่า Buffer เต็ม (ไม่เขียนทับข้อมูลเก่า) และหาก `tail == head` แสดงว่า Buffer ว่าง ไม่มีข้อมูลใหม่ให้



6. การบีบอัดข้อมูลแบบ Fixed-point ลงใน int64 (Pack / Unpack)

เนื่องจากสถาปัตยกรรม Ring Buffer ถูกออกแบบให้จัดเก็บข้อมูลขนาด 8 ไบต์ (int64_t) ต่อ 1 Slot การจัดเก็บข้อมูลเซนเซอร์หลายมิติพร้อมกันจึงจำเป็นต้องผ่านกระบวนการแปลงและบีบอัดข้อมูล (Data Packing) เพื่อให้สามารถบรรจุลงในพื้นที่ขนาดจำกัดได้อย่างมีประสิทธิภาพสูงสุด

6.1 หลักการออกแบบ Fixed-point Representation

การใช้ข้อมูลประเภท Floating-point (เช่น float หรือ double) โดยตรงใน Shared Memory ข้ามภาษาโปรแกรม (เช่น C++ กับ Python) อาจประสบปัญหาความคลาดเคลื่อนของการตีความรูปแบบทศนิยม (IEEE 754) โครงสร้างชุดนี้จึงเลือกใช้ Fixed-point Arithmetic โดยการแปลงค่าทศนิยมให้อยู่ในรูปตัวเลขจำนวนเต็มผ่านการคูณด้วย Scale Factor ที่กำหนดตายตัว ซึ่งช่วยรักษาความแม่นยำ (Precision) และเพิ่มความเร็วในการประมวลผลระดับฮาร์ดแวร์

6.2 การแพ็กข้อมูลเซนเซอร์ IMU (Roll, Pitch, Yaw)

ข้อมูลทิศทางการเคลื่อนไหว IMU ประกอบด้วยค่า Roll, Pitch และ Yaw ซึ่งแต่ละค่าจะถูกแปลงเป็นชนิดข้อมูล int16_t โดยกำหนด Scale Factor เท่ากับ 100 (รักษาความละเอียดทศนิยม 2 ตำแหน่ง เช่น 45.30 องศา จะถูกบันทึกเป็น 4530)

Bit Layout (64-bit)

- [บิตที่ 63 - 48]: ไม่ได้ใช้งาน (Unused / Padding 16 bits)
- [บิตที่ 47 - 32]: Yaw (16-bit signed integer)
- [บิตที่ 31 - 16]: Pitch (16-bit signed integer)
- [บิตที่ 15 - 0]: Roll (16-bit signed integer)

6.3 การแพ็กข้อมูลพิกัด GPS (Latitude, Longitude)

ข้อมูลพิกัดภูมิศาสตร์ประกอบด้วย Latitude และ Longitude โดยแต่ละค่าจะถูกแปลงเป็นชนิดข้อมูล `int32_t` (ให้ความละเอียดทศนิยม 6 ตำแหน่ง ซึ่งมีความแม่นยำสูงในระดับเซนติเมตรถึงเมตร)

Bit Layout (64-bit)

- [บิตที่ 63 - 32]: Longitude (32-bit signed integer)
- [บิตที่ 31 - 0]: Latitude (32-bit signed integer)

6.4 ข้อควรระวังในการแปลงข้ามภาษา (Cross-language Pitfalls)

เมื่อโปรแกรมภาษาอื่น เช่น Python ทำการอ่านค่า `int64` ดังกล่าว จำเป็นต้องจัดการเรื่อง Signed/Unsigned Casting และ Bitmask (& 0xFFFF หรือ & 0xFFFFFFFF)
ให้สอดคล้องกับขนาดบิตของแต่ละฟิลด์อย่างเคร่งครัด
มิฉะนั้นค่ามอดุลาร์หรือพิกัดซึ่งโลกใต้/ตะวันตกจะถูกตีความค่าทางคณิตศาสตร์ผิดพลาดทันที

7. การใช้งาน Shared Memory ฝั่งภาษา Go (camera.go) และ Raw Offset

สำหรับข้อมูลขนาดใหญ่ เช่น ภาพจากกล้องความละเอียด 640x480 (รูปแบบ RGB 3 ไบต์ต่อพิกเซล รวมขนาดภาพ 921,600 ไบต์) การใช้ Ring Buffer จะไม่เหมาะสม จึงเปลี่ยนมาใช้สถาปัตยกรรมแบบ Raw Byte Offset ร่วมกับ Atomic Ready Flag

เนื่องจากภาษา Go ไม่มีฟังก์ชัน shm_open โดยตรง จึงใช้งาน os.OpenFile() เปิดไฟล์ที่ /dev/shm/<name> และทำ syscall.Mmap() แทน ซึ่งให้ผลลัพธ์และประสิทธิภาพเหมือนกับฝั่ง C/C++

Offset (ไบต์)	ขนาด (ไบต์)	ชนิดข้อมูล	ความหมายและคำอธิบาย
0	8	uint64	Timestamp (เวลา UnixNano ของเฟรมภาพ)
8	4	uint32	Width (ความกว้างภาพ เช่น 640 พิกเซล)
12	4	uint32	Height (ความสูงภาพ เช่น 480 พิกเซล)
16	4	uint32	Ready Flag (0 = กำลังเขียน/ไม่พร้อม, 1 = พร้อมอ่าน)
20	921,600	byte[]	ข้อมูลภาพ RGB ดิบ (W * H * 3 ไบต์)

ลำดับการเขียนข้อมูลที่ถูกต้องสำหรับ Writer (Go): เขียนข้อมูลภาพลง offData ให้เสร็จก่อน -> เขียน Width, Height, Timestamp -> และเขียน Ready Flag = 1 เป็นลำดับสุดท้ายด้วย Atomic Store เพื่อให้ฝั่งอ่านมั่นใจได้ว่าข้อมูลสมบูรณ์แล้วจริงๆ

8. การเขียน-อ่านข้อมูลของ shared memory

8.1 ขั้นตอนการเตรียมและแพ็กข้อมูล (Data Packing)

ก่อนที่ข้อมูลจะถูกส่งไปยังหน่วยความจำแชร์
โปรเซสจะทำการประมวลผลภายในพื้นที่ส่วนตัวก่อนตามลำดับ

- การแปลงชนิดข้อมูล: นำค่ามุม Roll, Pitch และ Yaw ที่เป็นเลขทศนิยม (float) มาแปลงเป็นตัวเลขจำนวนเต็ม (int16_t) ผ่านฟังก์ชัน degToFixed() โดยคูณด้วย Scale Factor ที่กำหนดไว้เท่ากับ 100
- การแพ็กข้อมูลรวม: นำค่าจำนวนเต็มทั้ง 3 ตัวมารวมเป็นชุดข้อมูลขนาด 64 บิต (int64_t) ตัวเดียวด้วยฟังก์ชัน packIMU() ผ่านกระบวนการเลื่อนบิต (Bitwise Shift)

คำสั่งเขียนข้อมูลลง SHM จริง

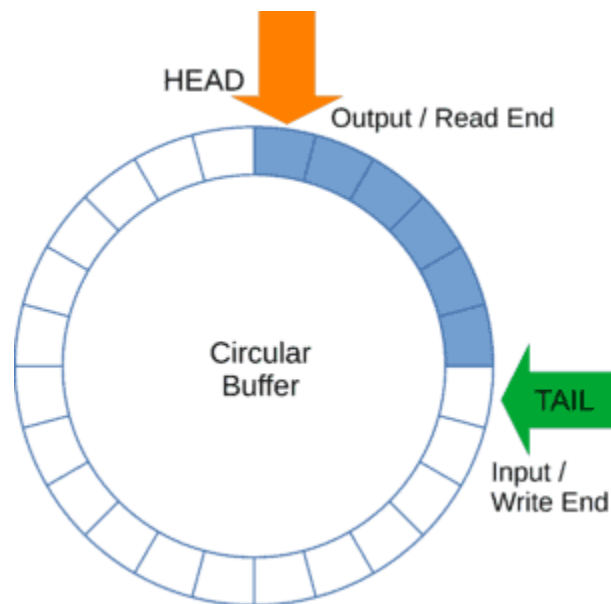
ภาษา C++

```
shared->imu_packet.push(packIMU(degToFixed(r), degToFixed(p), degToFixed(y)));
```

- กลไกการทำงานของ .push(): ข้อมูล 64 บิตที่แพ็กแล้วจะถูกนำไปบันทึกลงในอาร์เรย์ buffer[] ของ Ring Buffer ภายในโครงสร้าง SharedData ที่ถูกแมปไว้ด้วยระบบ mmap()
- การแชร์ข้อมูล: ทันทีที่มีการบันทึกสำเร็จ
ข้อมูลจะปรากฏในพื้นที่หน่วยความจำแชร์ระดับระบบปฏิบัติการ
ทำให้โปรเซสอื่นสามารถอ่านข้อมูลชุดนี้ได้ทันที

ความถี่ในการบันทึกข้อมูล (Sampling Rate)

- อัตราการทำงาน: ข้อมูลจะถูกเขียนลง SHM อย่างต่อเนื่องด้วยความถี่ 200 Hz (ทุกๆ 5 มิลลิวินาที) ภายในรูปการทำงานหลัก
- การควบคุมเวลา ใช้ฟังก์ชัน `clock_nanosleep` ร่วมกับ `CLOCK_MONOTONIC` และสถานะ `TIMER_ABSTIME` เพื่อกำหนดเวลาเป้าหมายแบบสัมบูรณ์ ช่วยป้องกันปัญหาความคลาดเคลื่อนของเวลา (Time Drift) ระหว่างรอบการทำงาน



8.2 ฝั่งอ่านข้อมูล (Reader) การดึงและถอดรหัสข้อมูล IMU จาก Shared Memory

กระบวนการฝั่งอ่านข้อมูลจะทำงานตรงข้ามกับฝั่งเขียน โดยมีขั้นตอนหลัก

การเชื่อมต่อและเข้าถึง Shared Memory

- ฝั่งอ่านจะทำการเชื่อมต่อและแมปเข้ากับพื้นที่ Shared Data เดิมผ่านฟังก์ชัน `initSharedMemory()` เพื่อเปิดใช้งานตัวชี้ (Pointer) ไปยังโครงสร้างข้อมูลที่แชร์ไว้
- ฝั่งอ่านจะคอยตรวจสอบสถานะของ Ring Buffer ผ่านตัวชี้ตำแหน่ง (Tail Index) เพื่อเช็คว่ามีข้อมูลชุดใหม่ถูกส่งเข้ามาในระบบหรือไม่

คำสั่งดึงข้อมูลจริง .pop()

คำสั่งหลักที่ทำหน้าที่ดึงข้อมูลออกจาก Shared Memory คือการเรียกฟังก์ชัน .pop() ของ Ring Buffer

ภาษา C++

```
int64_t raw_data;  
  
if (shared->imu_packet.pop(raw_data)) { ... }
```

- ฟังก์ชัน .pop() จะดึงข้อมูลขนาด 64 บิต (int64_t) ออกจากอาร์เรย์ buffer[] พร้อมทั้งอัปเดตตำแหน่ง Tail Index แบบปลอดภัย (Atomic Operation) เพื่อไม่ให้เกิดความขัดแย้งใน
- การเข้าถึงข้อมูลระหว่างโปรเซส การตรวจสอบสถานะหา Buffer ว่าง (Tail ชนกับ Head) ฟังก์ชันจะคืนค่า false ทำให้ฝั่งอ่านทราบว่ายังไม่มีข้อมูลใหม่ และสามารถข้ามรอบการทำงานไปได้โดยไม่เกิดข้อผิดพลาด

กระบวนการถอดรหัสข้อมูล (Unpacking & Conversion)

เมื่อดึงข้อมูล 64 บิต ออกมาได้สำเร็จ ฝั่งอ่านจะนำข้อมูลมาแปลงกลับเป็นค่ามุมที่ใช้งานได้จริงผ่านฟังก์ชัน:

การแยกสลายบิต (unpackIMU) นำค่า int64_t มาแยกสลายกลับเป็นค่าจำนวนเต็ม 16 บิต จำนวน 3 ตัว ได้แก่ Roll, Pitch และ Yaw ตามตำแหน่งบิตเดิมที่ถูกแพ็กไว้

การแปลงหน่วย Fixed-point เป็นองศา (fixedToDeg) นำค่าจำนวนเต็ม int16_t หารด้วย Scale Factor มาตรฐาน (หารด้วย 100) เพื่อแปลงกลับเป็นตัวเลขทศนิยม (float) ในหน่วยองศา พร้อมนำไปประมวลผลหรือแสดงผลต่อไป

9. ตัวอย่างการอ่านข้อมูลด้วยภาษา Python

สำหรับการเชื่อมต่อเพื่ออ่านข้อมูล SHM ด้วยภาษา Python สามารถใช้ Library เช่น mmap, struct และ posix_ipc โดยต้องจำลองการทำงานของ Bitwise Unpack และ Ready Flag Checking

ให้สอดคล้องกับโครงสร้าง C/C++ และ Go ทุกประการ ดังนี้:

- **การอ่าน IMU/GPS (Ring Buffer):** เปิดไฟล์ผ่าน `/dev/shm/<name>` และใช้ `mmap.mmap()` เพื่ออ่านโครงสร้าง Ring Buffer พร้อมคำนวณ Tail Pointer และ unpack ค่า Fixed-point กลับเป็นองศาหรือพิกัดจริง
- **การอ่าน Camera Frame:** ตรวจสอบ Ready Flag ด้วย atomic load แบบจำลอง (`struct.unpack_from`) และตรวจสอบ Timestamp เพื่อป้องกันการอ่านเฟรมซ้ำ ก่อนดึงข้อมูล RGB ไปประมวลผลต่อด้วย Pillow

10. ข้อควรระวังและแนวปฏิบัติที่ดี (Best Practices)

1. **Memory Layout & Alignment:** ทุกภาษาโปรแกรมที่เข้าถึง SHM ก่อนเดียวกันต้องตกลงโครงสร้าง Offset และ Struct Layout ให้ตรงกันทุกไบต์ ไม่มีการส่ง Schema ผ่าน SHM
2. **Endianness:** สถาปัตยกรรม x86-64 และ ARM ที่ใช้งานทั่วไปเป็น Little-endian การอ่านไบต์ต้องกำหนด Endianness ให้ถูกต้อง (เช่น ใช้ Format `<` ใน Python struct)
3. **Lifecycle & Cleanup:** ไฟล์ SHM จะยังคงค้างอยู่ใน `/dev/shm` แม้โปรแกรมหลักจะปิดตัวลง จนกว่าจะมีการเรียก `shm_unlink()` หรือรีบูตระบบ
4. **Permissions:** ตรวจสอบสิทธิ์ไฟล์เสมอ (`ls -la /dev/shm/`) เพื่อป้องกันปัญหา Permission Denied ระหว่างโปรเซสที่รันด้วย User ต่างกัน

11. คำแนะนำ

ก่อนเริ่มต้นพัฒนาโปรเจกต์ที่ใช้งาน POSIX SHM ร่วมกับระบบนี้ ผู้พัฒนาควรตรวจสอบรายการปฏิบัติ (Checklist)

ดังต่อไปนี้

- เลือกรูปแบบ SHM ให้ตรงกับประเภทข้อมูล
(Ring Buffer สำหรับข้อมูลเล็กถี่ / Raw Offset สำหรับภาพหรือก้อนข้อมูลใหญ่)
- ตรวจสอบชื่อ SHM ให้ถูกต้องตรงกันทุกตัวอักษร รวมถึงเครื่องหมาย '/' นำหน้า
- ห้ามใช้ O_CREAT และ ftruncate ในฝั่ง Reader ที่ทำหน้าที่อ่านอย่างเดียว
- ปฏิบัติตามกฎ SPSC (Single-Producer Single-Consumer) อย่างเคร่งครัดสำหรับ Ring Buffer
- ตรวจสอบ Ready Flag และ Timestamp ทุกครั้งก่อนอ่านข้อมูลภาพจาก Raw Offset
- ทดสอบระบบโดยรัน Writer ก่อน Reader เพื่อให้ระบบจัดการ Error กรณี Reader เปิดก่อนได้อย่างถูกต้อง